11. Dataflow, Graphs, and Parallel Symmetries

This section studies how streaming computation, graph structure, and symmetry interact when systems are executed under resource constraints. The central theme is that a dataflow network is not only a wiring diagram for computation, but also an object whose symmetries can be exploited for scheduling, load balancing, and architecture-aware transformation.

A useful performance summary for a streaming system is the error functional $\varepsilon(r)$, where r denotes a resource regime or runtime configuration. In this setting, $\varepsilon(r)$ may be read as a multi-objective measure combining throughput, latency, and deadline miss rate. High throughput alone is not sufficient: a configuration is acceptable only when latency remains bounded and deadline misses stay within the target envelope. Thus, the operational question is not merely whether a pipeline computes the right function, but whether it computes it at the right rate and with the right temporal guarantees.

Kahn process networks provide a canonical model for deterministic streaming computation. Their compositionality makes them a natural fit for the book's broader emphasis on closure: components can be connected by channels, and the resulting network can often be reasoned about by local properties of its parts. In the same spirit, streaming operads offer an algebraic language for composing stream processors with explicit interface arities. This viewpoint supports compositional scheduling, where the schedule of a large system is derived from the schedules of its subcomponents rather than imposed monolithically. The benefit is twofold: the semantics remain modular, and the implementation can be adapted to heterogeneous execution targets.

Graph structure becomes especially important when the dataflow graph admits nontrivial automorphisms. An automorphism of a pipeline preserves the connectivity pattern while permuting internal components, and such symmetries can be used to identify equivalent execution choices. In practice, graph automorphisms can expose load-balanced symmetries: if several subgraphs are structurally interchangeable, then work may be distributed across them in a way that preserves semantics while improving utilization. This is particularly relevant in replicated stages, fork-join patterns, and tiled computations, where symmetry can be turned into a scheduling resource.

The same structural ideas motivate fusion and fission laws. Fusion combines adjacent stages to reduce communication overhead and improve locality; fission splits a stage into parallel copies to increase throughput or match available hardware parallelism. These transformations are not purely syntactic optimizations: they must respect dependencies, buffering constraints, and the symmetry class of the underlying graph. When applied carefully, fusion and fission can be viewed as symmetry-preserving rewrites that move computation toward a more efficient realization without changing the observable stream behavior.

Hardware mapping makes these issues concrete. On GPUs and FPGAs, tiling is often the decisive step in obtaining good performance. A tile is a finite block of computation and data movement chosen to fit memory hierarchies, vector widths, or spatial fabric constraints. In a symmetric dataflow graph, tilings can be aligned with automorphism orbits so that repeated substructures are mapped uniformly across processing elements. This can reduce control complexity, improve cache or on-chip memory reuse, and make the implementation more predictable under scaling.

A useful way to organize the implementation is to treat the streaming kernel as a parameterized family indexed by symmetry transforms. One begins with an abstract stream specification, analyzes the graph-level symmetries, and then generates target-specific kernels for a chosen tiling or load-balancing strategy. In this workflow, the symmetry analysis is not an afterthought: it guides the choice of decomposition, the placement of buffers, and the granularity at which parallelism is exposed. The resulting build log should record the chosen $\varepsilon(r)$ metrics, the detected graph automorphisms, and the effect of fusion/fission passes on throughput and deadline miss rate.

Taken together, these ideas show that streaming systems are best understood as compositional graphs with exploitable symmetry. The algebraic perspective clarifies when schedules can be derived locally, while the graph-theoretic perspective identifies equivalence classes of execution strategies. The practical outcome is a disciplined route from abstract dataflow specification to parallel implementation on modern hardware, with performance measured against explicit latency, throughput, and deadline constraints.